Software Engineering Department

Braude College of Engineering

Nappi - Digital Twin Based Platform for Monitoring and Improving Baby Sleep

Project Code: 25-2-D-22

Capstone Project Phase B - 61999

March 2026

Ofer Elzara

Oran Efroni

Supervisor:

Mr. Uzi Rozen

Git repository link:

Link

Simulation Video

Link

Table of Contents

Overview 4

1. Introduction 4

2 Research and Background Summary 5

2.1 Research Motivation 5

2.2 Theoretical Limitations of Current Monitoring Systems Summary 5

2.3 Theoretical Background 6

2.3.1 Digital Twin Approach 6

2.3.2 Machine Learning Models 6

2.3.3 Large Language Models 6

2.3.4 Asynchronous Database Management 6

2.3.5 FastAPI 7

3 General System Description 7

3.1 System Capabilities 7

3.1.1 Multi-Sensor Monitoring and Data Persistence 7

3.1.2 Sleep Session Tracking 8

3.1.3 Alerting System with Real-Time Delivery 8

3.1.4 Last Sleep Summary 8

3.1.5 Trend Analysis and Schedule Prediction 8

3.1.6 Sleep Analytics and Historical Reporting 9

3.1.7 Correlation-Based Awakening Analysis with AI Insights 9

3.1.8 Context-Aware AI Chat Assistant 9

3.2 Primary User Workflow 9

3.3 Target Persona and Stakeholder 10

3.3.1 Target Persona 10

3.3.2 Stakeholder 10

4 System Architecture 11

4.1 Architecture Overview 11
4.1.1 Backend 11
4.1.2 Frontend 12
4.1.3 IoT Devices 13
4.1.4 Database 13
4.2 Main System Flow 14
4.3 Key Modules 14
4.3.1 Scheduler and Data Miner 14
4.3.2 Sleep State Manager 14
4.3.3 Alert Service 15
4.3.4 Correlation Analyzer 15
4.3.5 Sleep Patterns and Schedule Predictor 15
4.3.6 Optimal Stats 15
5 Software Development Process 15
5.1 Development Stages 15
5.2 Work Division 16
5.3 Decisions & Constraints 16
6 Results 18
7 Evaluation 19
8 Conclusions & Learned Lessons 19
9 References 20
10 Appendices 21
11 User Manual 22
12 Maintenance Guide 41
13 Caregiver Persona Survey 51

Overview

Nappi is a 'digital twin' inspired infant sleep monitoring platform that helps parents reduce guesswork by turning home sensor readings and sleep/wake events into correlation-based insights and likely contributing factors. The system uses an M5Stack micro controller & environmental sensors, a FastAPI backend and database, alongside a React dashboard, enabling real-time monitoring, alerts, and long-term trend tracking. It ingests and aggregates environmental measurements (temperature, humidity, noise), generates daily summaries and "optimal conditions" insights, and includes AI-generated insights for awakenings based on recent sensor-change analysis, along with a context-aware chat assistant that answers parent questions using the baby's full context (profile, notes, patterns, and history).

1. Introduction

Sleep is a cornerstone of healthy brain and body development in the first years of life. Yet an estimated 20-30 percent of infants experience significant nighttime awakenings that disrupt restorative sleep cycles [1]. Chronic sleep fragmentation has been linked to slowed cognitive growth, weaker immune response, and increased parental stress [2]. Many environmental factors can nudge a baby from deep sleep to wakefulness, but these influences are rarely tracked in real time inside the home. Although an expanding market of "smart" baby monitors promises help, most current devices focus on a single signal (audio, video, or wearable pulse oximetry) and provide limited, event by event alerts [3]. Parents still lack access to structured, data-driven insights into how and why sleep changes over time. Clinical sleep studies require in-clinic polysomnography, long waitlists, and costs often around $1,000–$3,000. For instance, "sleep resolutions" offers such services (link). These factors represent barriers that leave many families without access to actionable guidance.

To address these gaps, we thought about a Digital Twin based approach. By collecting real-time data over an individual infant, our program could detect correlations between environmental changes and wake up events and track long term sleep trends (improvement or decline) for each baby.

Our motivation is personal and based on a close-relative parent persona. We focused our proof of concept on the everyday challenges experienced by new parents in our immediate circles, where sleep disruptions often lead to months of trial-and-error with traditional baby monitors that primarily emphasize safety rather than insight. This repeated frustration inspired us to build a more data-driven solution aimed at helping families better understand and improve infant sleep. We believe this platform can provide statistical based recommendations and allow parents access to relevant data of their infant sleep, offering them actionable insights that were previously confined to clinical laboratories.

2 Research and Background Summary

This chapter examines the importance of infant sleep, the common biological and environmental factors that influence it, and the challenges caregivers face in monitoring sleep effectively. It also reviews relevant real-time baby monitoring systems and sensor technologies, which inform the development of our proposed 'digital twin' based solution for analyzing sleep behavior and providing data-driven support to caregivers.

2.1 Research Motivation

Phase A framed infant sleep as a high-impact health and family well-being domain: 20–30% of infants experience significant night awakenings [1], and repeated sleep fragmentation is associated with negative developmental and parental outcomes [2].

A key market gap identified in the literature review is that many "smart baby monitors" still behave as single-signal tools (audio/video/wearable), producing event-by-event notifications rather than structured, cross-factor insight [3]. As a result, parents remain responsible for guessing what caused their baby to wake up, whether environmental change, discomfort, or a normal sleep cycle.

Phase A also used feedback to refine what caregivers consider useful: evidence-based suggestions about why wakeups occur, alerts that remain focused on high-importance situations rather than noise, and strong assurances of personal and family privacy.

Finally, the research motivation explicitly connected to known environmental sleep disruptors. Phase A highlighted that temperature, humidity, and noise are practical in-home variables with recognized safe/ideal ranges, making them suitable targets for continuous monitoring.

Therefore, the Nappi concept was defined to combine continuous multi-parameter sensing with real-time alerting and interpretation, so that caregivers receive privacy-aware, data-driven explanations.

2.2 Theoretical Limitations of Current Monitoring Systems Summary

Real-time monitoring plays a crucial role in infant care, providing continuous oversight that can detect problems as soon as they arise. Infants cannot verbalize discomfort or symptoms, so real-time sensor systems can alert caregivers to issues like irregular breathing or vital signs that may otherwise go unnoticed. By continuously tracking an infant's physiological signals and environment, these systems help ensure immediate intervention when needed and give parents peace of mind.

2.3 Theoretical Background

2.3.1 Digital Twin Approach

Digital twin technology provides a structured way to represent a real-world entity as a continuously updated virtual model, enabling analysis and prediction based on real-time data. In the Nappi context, this concept supports integrating multiple sleep-related signals into a single evolving "baby profile," making it possible to summarize trends over time and generate personalized insights and recommendations rather than isolated monitoring events.

2.3.2 Machine Learning Models

In general, machine learning models for time-series health monitoring aim to learn patterns from continuous streams. Their strengths are pattern discovery and personalization, their main practical limitations are data volume/quality, label scarcity, and explainability (parents need plausible reasons, not only predictions).

Phase A positioned Nappi as a system that prepares for ML-based personalization by continuously collecting structured signals and linking them to sleep outcomes, describing this as groundwork for "future integration of additional sensor types, advanced analytics, or AI-driven features."

This aligns with our project's digital-twin framing, in which the infant and surrounding environment are modeled as a continuously updated representation that can be analyzed to identify patterns and anticipate disturbances.

2.3.3 Large Language Models

Applying Large Language Models (LLMs) are useful in this domain because parents often need natural-language interpretation: turning context and measurements into a readable explanation "what likely woke my baby?" and actionable suggestions. The core limitation is reliability: LLM outputs can be sensitive to prompt framing and must be constrained with context, structure, and low-variance settings.

2.3.4 Asynchronous Database Management

Asynchronous DB management is a practical requirement in systems that must ingest sensor events, serve dashboards, and stream alerts without blocking. It enables the system to handle many simultaneous sensor events and user requests without blocking, but it requires careful control of database sessions and transactions to avoid stale connections, race conditions, and inconsistent writes.
This aligns with the Phase A architectural intent that the backend should be the only component that communicates with the DB and exposes APIs to the rest of the system.

2.3.5 FastAPI

FastAPI is an async-first Python framework suited for Nappi's need to combine:

- structured REST endpoints (dashboard, stats, chat),
- validation and typed contracts,
- integration with background tasks and real-time delivery.

Phase A already framed the backend as a FastAPI layer responsible for ingesting sensor JSON, running sleep/awake rules, triggering alerts, and correlating environment changes.

3 General System Description

3.1 System Capabilities

Our sleep monitoring platform designed to help caregivers understand what happened during sleep and what may have caused awakenings, using a combination of sensor data, automated analysis, and AI-assisted explanations.

At a baseline level, the system supports the core capabilities expected from a modern monitoring solution: onboarding a caregiver account, registering a baby profile, presenting current nursery conditions in a dashboard, summarizing the latest sleep session, and maintaining a history of alerts and events.

Beyond the basic features, we extended Nappi with more data-based analysis. Instead of only presenting raw measurements, the system also looks for repeating patterns, checks whether changes in the environment happen around awakenings, and suggests possible adjustments using the baby's recorded history together with general age-related sleep guidelines.

3.1.1 Multi-Sensor Monitoring and Data Persistence

The system continuously collects and stores nursery signals as structured, time-stamped records. Supported signals include:

- Temperature readings
- Humidity readings
- Noise level readings

All readings are stored in the database for later analytics, enabling caregivers to review trends across hours, days, and weeks rather than relying only on "live view". In addition, the sensor pipeline was designed in a modular way, so new sensors can be added with minimal changes (mainly by defining the new sensor type and plugging it into the existing ingestion and storage flow).

3.1.2 Sleep Session Tracking

Nappi maintains a consistent sleep timeline by tracking sleep sessions through sleep-start and sleep-end events, and by keeping an updated "current sleep status" for each baby. The system supports:

- Recording sleep start and sleep end timestamps
- Calculating sleep duration for each session
- Preventing rapid "flip-flopping" in status using a cooldown mechanism after manual or detected changes

This capability creates a reliable session history that later analytics and insights can use.

3.1.3 Alerting System with Real-Time Delivery

The system creates alerts for key events (baby wake-up and sensor threshold warnings) and delivers them in real time. It supports:

- Storing alert history per user, with baby id
- Read/unread state with an unread counter
- Live updates to the frontend (SSE)This capability creates a reliable session history that later analytics and insights can use
- Severity levels for prioritization
- Web push notifications when the app is not open

To avoid noise, the system does not alert on sleep-start or on manual intervention actions.

3.1.4 Last Sleep Summary

Instead of forcing the user to interpret raw events, the system provides a "last sleep summary" view that consolidates the most recent sleep session into a readable snapshot, including:

- Start time and end time
- Total sleep duration
- Awakening count (when relevant events exist)
- Last room metrics alongside the sleep summary for context

3.1.5 Trend Analysis and Schedule Prediction

To support parental planning, our system provides forward-looking analytics:

- Weekly and monthly sleep trend summaries
- Age-aware comparisons that interpret sleep amounts relative to typical expectations
- Schedule prediction for the next likely sleep window based on historical patterns and wake window guidance

3.1.6 Sleep Analytics and Historical Reporting

The system includes a dedicated analytics area that transforms stored sensor events and sleep sessions into interpretable statistics and visual patterns. Key analytics features include:

- Daily sleep duration reporting
- Visualization of historical sleep sessions and sleep distribution
- Schedule prediction for the next likely sleep window based on historical patterns and wake window guidance
- Personalized "optimal conditions" estimation (temperature, humidity, noise) based on the baby's best sleep sessions

3.1.7 Correlation-Based Awakening Analysis with AI Insights

The system can generate an explanation for a wake-up event by analyzing sensor readings shortly before the awakening. It supports:

- Selecting the latest awakening event (or a specific event by id) for analysis
- Pulling sensor data from a configurable time window before the wake-up
- Calculating percentage changes per parameter and filtering only "significant" changes (above a configured threshold)
- Generating an AI insight (regular text, and an optional structured version) and saving the result as a correlation record in the database sessions

In addition, when a wake-up is recorded by the sensor, the system generates a short AI insight sentence and attaches it to the awakening event.

3.1.8 Context-Aware AI Chat Assistant

The system includes a chat screen that lets the caregiver ask questions and receive an AI response based on the baby's stored data. The AI chat assistant build the response context from stored data: baby profile, notes, optimal stats, recent awakenings (with insights), recent correlations, daily summaries, monthly sleep patterns, and the latest room readings

3.2 Primary User Workflow

The caregiver first creates or selects a baby profile and optionally adds notes that describe the baby's routine. When the baby goes to sleep, monitoring starts and the system begins collecting and storing relevant signals in case of unusual reading or general notes, user receive alerts. Caregiver can review a summarized timeline of the sleep session and key events and can inspect awakening events to see the surrounding context and suggested contributing factors. If needed, the caregiver can ask the AI

assistant questions about the night and then apply small routine or environment adjustments and track changes across nights.

3.3 Target Persona and Stakeholder

3.3.1 Target Persona

As a Proof of Concept, we intentionally focused Phase B on the same primary persona defined in Phase A, because our early project motivation and system goals were built around this user's daily pain points: a time constrained caregiver of a 7 month infant with frequent night awakenings who needs an easy way to understand what might happened and what to try.

This focus shows in Phase B by putting caregiver features first - simple night summaries, correlation-based awakening insights, alerts, and an AI chat based on the baby's data.

To validate and refine these needs, we conducted a survey with our persona focused on infant sleep challenges and common monitoring habits, and attitudes toward AI support in baby monitoring (trust, usefulness, concerns). The full survey is attached in the Appendices.

3.3.2 Stakeholder

Caregivers –main stakeholder of this PoC, since the system was designed around their need to understand infant awakenings and get simple, practical support during daily use (as described in Section 3.3.1).

Researchers/clinicians - The PoC can collect structured "sleep & environment" history (events with temperature/humidity/noise context), which may support future studies on what conditions tend to appear around awakenings and how patterns change over time - while keeping privacy in mind by storing only statistical data and summaries over time (not photos or recorded media).

Industry/product direction - The PoC points to a gap we saw for our persona: many baby monitors mainly stream video/audio or send basic alerts, while caregivers often want a simple explanation of what might have triggered an awakening and what to try next. Nappi aim to explores this direction and demonstrate a possible new value focus for baby monitors.

4 System Architecture

The system architecture was initially defined in Phase A based on the system characterization, constraints, and goals identified at that stage. During Phase B, it was refined to reflect the practical design decisions and adjustments that emerged throughout development.

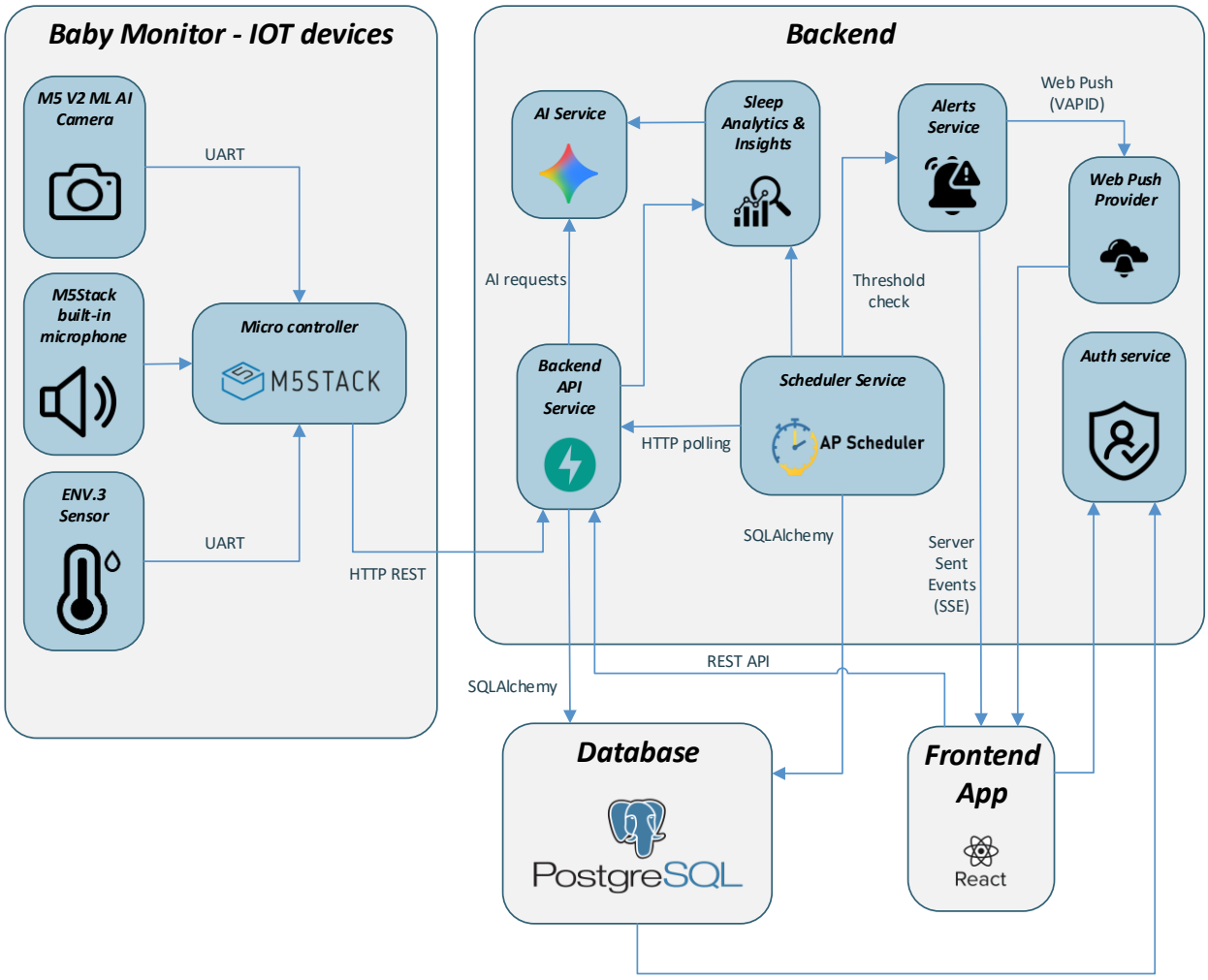


Figure 1. Nappi System Architecture

4.1 Architecture Overview

4.1.1 Backend

The backend is implemented in Python using FastAPI, an async-first web framework that handles concurrent sensor ingestion, dashboard requests, and real-time alert delivery without blocking. It exposes a set of RESTful endpoints organized into four routers: monitoring endpoints for room and sleep data, sensor event endpoints for sleep-start/sleep-end signals from the M5Stack, a statistics router for trend analysis and AI insights, and a chat router for context-aware AI responses.

Authentication is handled through a dedicated auth router supported by an auth manager service that issues and validates tokens, ensuring baby data is accessible only to the associated caregiver account. Background processing is managed by APScheduler, which is initialized at application startup under the app's lifespan context. The scheduler runs three jobs: periodic sensor polling via an HTTP client (data_miner.py) that reaches out to the M5Stack only while a baby is sleeping, a daily summary job that aggregates raw readings into per-day records, and an optimal stats job that computes weighted-average environmental conditions based on sleep quality history. Scheduled jobs interact directly with the database and, in the case of sensor polling, also invoke the alert service to check environmental thresholds after each collection cycle.

AI capabilities are powered by Google Gemini and are used in three distinct flows: generating a short insight attached to each awakening event, producing structured correlation analysis (identifying which environmental changes likely preceded a wake-up), and answering caregiver questions in the chat assistant using the baby's full stored context. FastAPI was selected over alternatives due to its native async support, Pydantic-based validation, and straightforward integration with background task management.

4.1.2 Frontend

The frontend is a single-page application built with React and TypeScript, bundled with Vite. It communicates with the backend exclusively through a typed API layer organized into dedicated modules for authentication, sleep data, room metrics, alerts, chat, and statistics. Real-time alert delivery is handled via Server-Sent Events (SSE), where the frontend maintains an open ‘EventSource’ connection to receive push updates without polling. For web push notifications when the app is not open, a service worker (sw.js) intercepts incoming push messages from the Web Push provider.

Unlike the Flutter-based approach considered in Phase A, a web application was chosen as it requires no installation on the caregiver's device and is accessible from any browser. React was selected for its widespread adoption and component model, which suited the dashboard-heavy interface. TypeScript was added to enforce type safety across API calls and reduce integration errors with the backend.

4.1.3 IoT Devices

The IoT layer consists of an M5Stack core 2 microcontrollers connected with M5GO2 secure element, connected to three peripherals: an ENV.3 sensor for temperature and humidity, the M5Stack's built-in microphone for ambient noise level, and an M5 V2 AI Camera module.

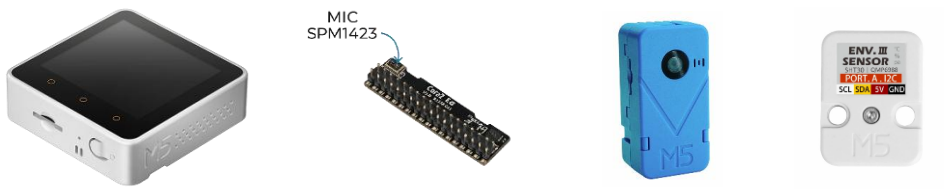


Figure 2. Nappi IoT edge hardware (left to right): M5Stack Core2 microcontroller & built-in SPM1423 digital microphone, M5 Unit V2 ML AI Camera module, and ENV.3 temperature & humidity sensor.

Sensor readings are assembled on the device into a JSON payload and made available via a HTTP endpoint, which the backend scheduler polls at a configurable interval. In parallel, the M5Stack independently sends HTTP POST requests to the backend when it detects a sleep-state change (sleep-start, sleep-end, or baby-away), allowing the backend to begin or stop sensor collection accordingly. The camera module was used during Phase B to develop a TinyML-based sleep/wake classifier trained using Google Teachable Machine, implemented as a lightweight on-device proof of concept. We chose this direction after reviewing prior video-based infant sleep/wake studies, which show that ML classifiers can work well using short video clips [4]. Also, some models used visual cues such as posture and (when visible) face/eye activity.

4.1.4 Database

The system uses PostgreSQL as its persistent store, accessed through SQLAlchemy with async support via asyncpg. The database schema covers seven primary concerns: user accounts, baby profiles, raw sensor readings (retained short-term and cleared after daily summarization), daily summaries, optimal stats per baby, awakening correlation records with AI insights, and alerts. The backend is the sole component that communicates with the database, enforcing a clear data boundary. PostgreSQL was selected for its reliability with relational data that has defined ownership rules (a user owns a baby. a baby owns its alerts and sessions).

4.2 Main System Flow

A high-level overview of the system's activity flow is presented in Figure 3, tracing the sequence of operations across the four main layers: the M5Stack IoT device, the backend API, the database, and the frontend web application.

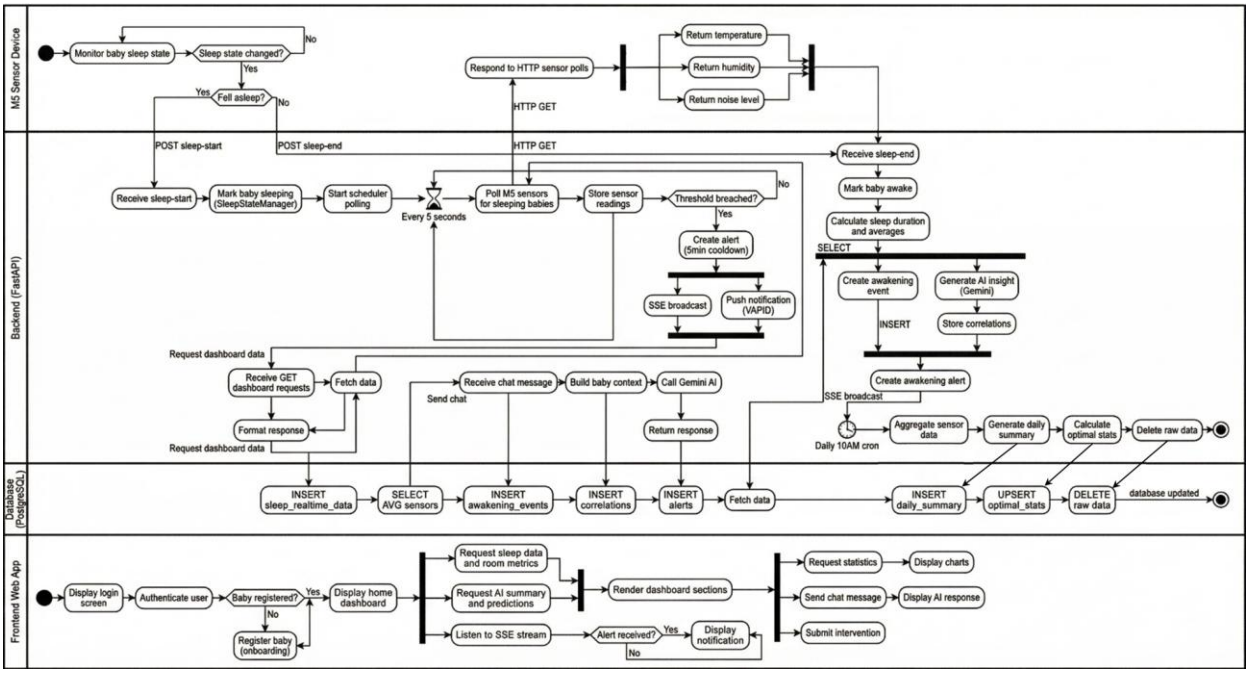


Figure 3. Nappi System Activity Flow Diagram

4.3 Key Modules

4.3.1 Scheduler and Data Miner

The scheduler runs three background jobs: periodic sensor polling (only for sleeping babies), a daily summary job, and an optimal stats job. The data miner implements the HTTP client that reaches out to the M5Stack's local endpoint to fetch the latest sensor readings and is only invoked when at least one baby is registered as sleeping.

4.3.2 Sleep State Manager

Maintains an in-memory registry of which babies are currently sleeping. It is updated by sleep-start, sleep-end, and baby-away events received from the M5Stack, and is consulted by the scheduler before every sensor poll cycle to determine which babies require active data collection. It also manages a 20-minute intervention cooldown, during which sensor-reported events are ignored following a manual parent override.

4.3.3 Alert Service

Creates and stores alerts for awakening events and environmental threshold breaches. It manages an SSE connection registry, broadcasting alerts in real time to any connected frontend client. It also delegates to the push service to deliver web push notifications when the frontend is not open.

4.3.4 Correlation Analyzer

Triggered on each sleep-end event. It retrieves sensor readings from the hour preceding the awakening, calculates percentage changes per parameter, filters only significant changes above a configured threshold, and sends the result to Gemini to generate a structured insight. The output is stored as a correlation record linked to the awakening event.

4.3.5 Sleep Patterns and Schedule Predictor

The patterns module clusters historical sleep sessions by start-time proximity to identify recurring sleep windows (morning nap, afternoon nap, night sleep). The schedule predictor uses these clusters alongside age-based wake window guidelines to estimate the next likely sleep window for the baby.

4.3.6 Optimal Stats

Runs daily after the summary job. It calculates a weighted average of environmental conditions across all historical sessions, where the weight for each day is inversely proportional to the number of awakenings recorded. The result represents the conditions under which that specific baby has historically slept best.

5 Software Development Process

Phase B focused on translating the Phase A design into a working system under real constraints. This chapter documents the actual development stages, the decisions that shaped the implementation, the main issues we faced, and the final results.

5.1 Development Stages

A. **Foundations:** Set up an initial Backend, Frontend skeleton (FastAPI structure, basic endpoints, mock pages intended to be implemented).
B. **Hardware Inspection:** Met with our supervisor and refined the hardware plan. Based on availability and practicality, we moved from an initial Raspberry Pi direction to M5Stack as the main device, and selected sensors that were accessible and suitable for our measurements.

C. **Database Design:** Held a team planning meeting to define what data must be stored, how it flows through the system, and which tables are required. This stage shaped the database structure and the API payloads used later.
D. **Incremental Service Build:** Gradually added the core service logic from sleep session handling towards analytics jobs and scheduled jobs. Finalized with AI integration.
E. **End to End integration:** Performed an integration across the device layer, backend APIs, database persistence, service logic, and frontend presentation. This stage focused on addressing integration defects and edge cases. Finalizing the project for demonstration video and supporting documentation (User Guide and Maintenance Guide).

5.2 Work Division

The project work was split to keep clear ownership between the physical/UX layer and the server/data layer, while coordinating on shared flows and integration.

Student A (Frontend + IoT):

- Developed the frontend UI, including core screens and user interactions.
- Built and configured the M5Stack + sensors setup (wiring, readings collection).
- Handled data transmission reliability from the device to the system.

Student B (Backend + Database):

- Developed the backend services and APIs.
- Designed and maintained the database schema and persistence logic.
- Implemented server-side flows for storing incoming data and exposing it for analytics and display.

5.3 Decisions & Constraints

During implementation, several Phase A assumptions were refined due to hardware availability, performance limits, and privacy requirements:

- **Hardware selection**: Following supervisor feedback, the initial plan to use a Raspberry Pi was replaced with an M5Stack microcontroller. The Raspberry Pi was considered overreach for the project's scope, and its GPIO-based sensors require manual soldering. The M5Stack offered a simpler, modular sensor ecosystem better suited to the project's needs.

- **Sleep/Awake Detection - Shift to TinyML:** Phase A approach of using a NoIR camera with Dlib-based EAR(eye aspect ratio) detection proved problematic for two reasons: a baby's eyes are frequently not visible during sleep, and installing the required computer vision libraries on the M5Stack was impractically slow and performed poorly. After reviewing alternative ML approaches, we leveraged the M5 Unit V2 AI Camera's on-device inference capability and trained a lightweight classifier using Google Teachable Machine on our own persona's infant collected data. This is a naive solution, but it served as a sufficient PoC demonstration for this component.

Figure 4. Sample Persona's baby training images for the TinyML sleep-state classifier (asleep/no baby found/awake)

- **Privacy-First Camera Design:** A core constraint from Phase A was keeping sensitive data local. To achieve this, the camera feed never leaves the device - the camera only outputs a classification result (asleep / awake / away), ensuring no video or image data is transmitted or stored anywhere in the system.
- **Absence of Real Baby Sleep Data:** During the period between Phase A and Phase B, we contacted several organizations to obtain a real infant sleep dataset for validation. Those that responded declined due to privacy restrictions. As a result, the project scope was adjusted from a validated product to a PoC, and mock data was used for system validation and demonstration.

- **Hardware Enclosure & Mounting:** More iterations were required than initially anticipated. An early attempt involved collaboration with an industrial designer who produced a 3D-printed casing, however the print failed mid-process due to structural design flaws, and the approach was abandoned. Following this, the decision was made to assemble the enclosure manually. The task was broken into two sub-problems: a mounting handle that could attach securely to a crib, and a casing that could house all the hardware components (M5Stack Core2, ENV.III unit, and camera) while remaining safe and appropriate for a nursery environment. Several iterations were explored for each part before arriving at the final assembly shown below.

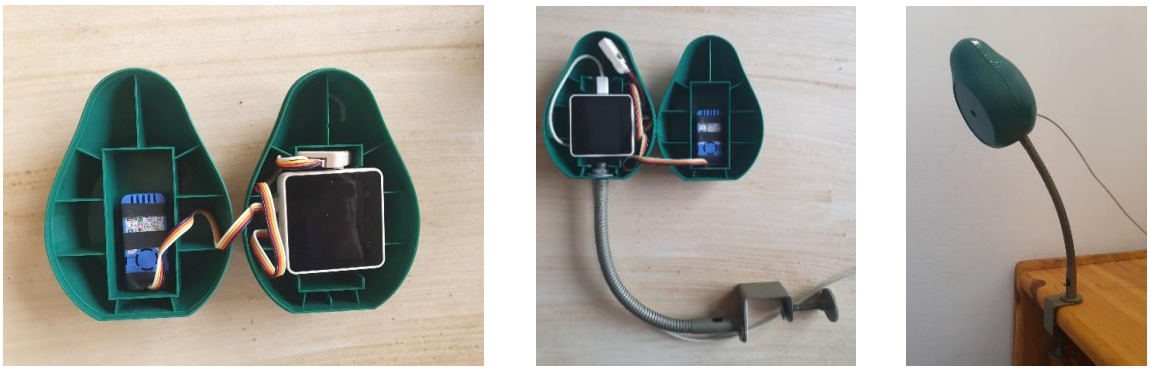


Figure 5. Physical hardware enclosure final design

6 Results

Phase B produced a functioning end-to-end PoC of the Nappi platform. The full system was integrated and demonstrated across all layers: an M5Stack device collecting environmental readings and reporting sleep-state events, a FastAPI backend handling data ingestion, scheduled analytics, and real-time alert delivery, and a React web dashboard presenting live room conditions, sleep history, and AI-generated insights to the caregiver.

Due to the constraints described above - primarily the absence of a real infant sleep dataset, the focus shifted toward building a solid, extensible platform rather than a fully validated product. The system is designed so that, given a real dataset, it can transition from PoC to product with minimal structural changes, and serve as a foundation for more advanced analytics, including machine learning based correlation discovery between environmental conditions and sleep outcomes

7 Evaluation

Evaluation in Phase B was done through various PoC tests. focusing on context quality, edge reliability, and user-visible responsiveness.

Evaluation Name	Methodology	Requirement	Result
AI Context Accuracy	10 fixed caregiver prompts, manual check	≥ 80% answers are relevant and baby-specific	90%
Alert Delivery Latency	Trigger alerts, measure backend → UI (SSE)	≤ 2 seconds	✓
Statistics Screen Render Time	Measure statistics page graphs UI refresh time after changing sensor or date range	≤ 2 seconds per chart	✓
Real-time Monitoring	Run M5Stack demo mode (simulated sleep cycle) and verify logger and database output	Sensors data polled every 5s with live SSE streaming	✓

8 Conclusions & Learned Lessons

1. Alerts are delivered in real time using SSE (and optionally Web Push). This supports a realistic “dashboard open” scenario, but it also introduced connection handling and edge cases (disconnects, reconnects, multiple tabs), and we had to keep alert logic “soft” and not spammy.
2. The M5Stack Core2 proved capable of running the entire embedded side of the system - reading three sensors, running a non-blocking HTTP server, debouncing camera verdicts, and maintaining WiFi, all within a single MicroPython event loop at a 500 ms tick rate. However, operating everything on the same thread meant that a slow sensor read or a hanging HTTP request would stall the entire loop. In a more mature implementation, separating the HTTP server into a background thread would improve robustness, though MicroPython's threading support on the ESP32 introduced its own instability during development, which is why the single-threaded architecture was ultimately chosen.
3. The backend's use of raw SQL via SQLAlchemy's text() rather than an ORM gave full control over query structure, which was important for the more complex aggregation queries (sleep patterns, daily summaries, trend analysis). The tradeoff is that query construction and parameter handling require more care, and several schema changes during development - such as the removal of the ‘anomalies’ column that was never populated - reflected situations where features were partially planned but easily edited.

9 References

[1] Jodi A. Mindell, Brett Kuhn, Daniel S. Lewin, Lisa J. Meltzer, Avi Sadeh, “Behavioral treatment of bedtime problems and night wakings in infants and young children: An American Academy of Sleep Medicine review.” Sleep, vol. 29, no. 10, 2006, pp. 1263-1276.

[2] Elaine KH Tham, Nora Schneider, Birit FP Broekman. “Infant sleep and its relation with cognition, growth: a narrative review.” National Library of Medicine, Nature and Science of Sleep, vol. 9, 2017, pp. 135-149.

[3] Helen L. Ball & Alice-Amber Keegan “Digital health tools to support parents with parent-infant sleep and mental well being”. npj Digital Medicine, vol. 5, 2022.

[4] Shayda Moezzi et al. “Classification of Infant Sleep–Wake States from Natural Overnight In-Crib Sleep Videos”. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops, 2025.

References Used in Application Development

[1] Franco, P. et al. "Ambient Temperature is Associated with Changes in Infants' Arousability from Sleep." SLEEP, vol. 24, no. 3, 2001, pp. 325–329.

[2] Elaine KH Tham, Nora Schneider, Birit FP Broekman. "Infant sleep and its relation with cognition and growth: a narrative review." Nature and Science of Sleep, vol. 9, 2017, pp. 135–149.

[3] Bhargava, S. "Diagnosis and Management of Common Sleep Problems in Children." Pediatrics in Review, vol. 32, no. 3, 2011, pp. 91–99.

[4] Sarah C Hugh, Nikolaus E Wolter, Evan J Propst, Karen A Gordon, Sharon L Cushing, Blake C Papsin. "Infant Sleep Machines and Hazardous Sound Pressure Levels." Pediatrics, vol. 133, no. 4, 2014, pp. 677–681.

[5] World Health Organization. Night Noise Guidelines for Europe. WHO Regional Office for Europe, 2009.

[6] Hirshkowitz, M. et al. "National Sleep Foundation's Sleep Time Duration Recommendations." Sleep Health, vol. 1, no. 1, 2015, pp. 40–43.

[7] Sadeh, A. et al. "Sleep and Sleep Ecology in the First 3 Years." Journal of Sleep Research, vol. 18, 2009, pp. 60–73.

[8] Adams, EL, Savage, JS, Master, L, Buxton, OM. "Time for Bed! Earlier Sleep Onset is Associated with Longer Nighttime Sleep Duration During Infancy." Sleep Medicine, vol. 73, 2020, pp. 58–63.

[9] NHS UK. "Reduce the Risk of Sudden Infant Death Syndrome." https://www.nhs.uk/conditions/baby/caring-for-a-newborn/reduce-the-risk-of-sudden-infant-death-syndrome/

[10] U.S. Environmental Protection Agency. "Care for Your Air: A Guide to Indoor Air Quality." https://www.epa.gov/indoor-air-quality-iaq/care-your-air-guide-indoor-air-quality

Appendices

User Manual

1. Introduction

Welcome to Nappi, a modern web application designed to help parents understand and improve their baby's sleep. Nappi combines real-time room monitoring (temperature, humidity, and noise), detailed sleep tracking, smart alerts, and AI-powered insights, all in one place.

This manual walks you through every feature of the app, from your first registration all the way to reading AI sleep analysis charts. Screenshots are included throughout to guide you step by step.

1.1. What Nappi Does

- Tracks when your baby falls asleep and wakes up, automatically via sensors or through your manual input.
- Monitors room conditions (temperature, humidity, noise) and identifies what your baby sleeps best in.
- Sends real-time push notifications when something changes, baby woke up, noise spike detected, and more.
- Shows rich analytics: sensor trend charts, daily sleep duration, a 24-hour sleep pattern clock, and weekly AI analysis.
- Provides an AI chat assistant you can ask anything about your baby's sleep, using notes you add about your baby's health and preferences.

1.2. How to Access Nappi

Nappi runs in any modern mobile or desktop browser. No app-store download is required. Simply navigate to the Nappi web address provided to you and the app will behave like a native mobile application.

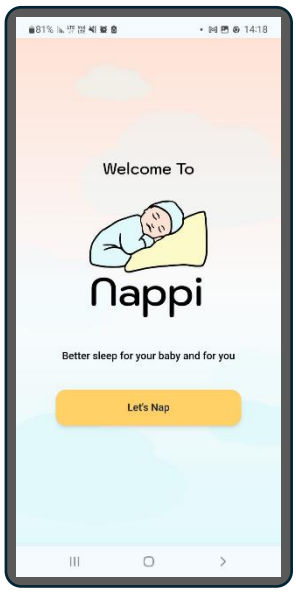
Note: For the best experience, open Nappi in Chrome or Safari on your smartphone and use the browser's 'Add to Home Screen' option to install it as an app icon.

2. Getting started

2.1. Welcome Screen

When you open Nappi for the very first time, you will see the Welcome screen. It displays the Nappi logo and the tagline 'Better sleep for your baby and for you', along with a large 'Let's Nap' button.

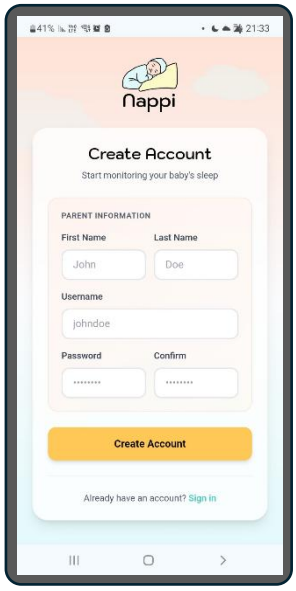
Tap 'Let's Nap' to proceed to the Sign In page.



New user: If you do not yet have an account, tap 'Sign up' on the following Sign In page to create one.

2.2. Creating a New Account

If you are a new user, tap 'Sign up' on the Sign In page. You will be taken to the sign up page, which is divided into two sections: Parent Information and Baby Information

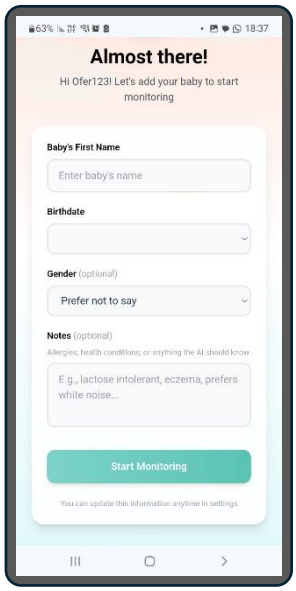


<u>Parent Information</u>: First & last name, username, password, password confirmation. Press Create Account.

2.3. Onboarding

Adding Your Baby After Registration

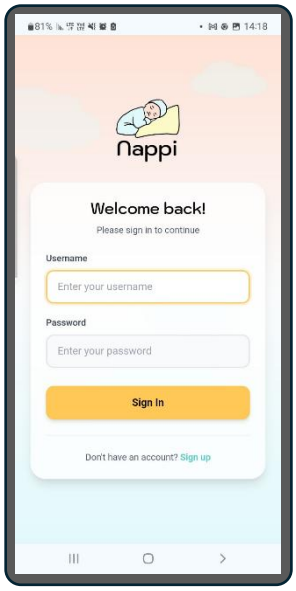
after registration Nappi will take you to a dedicated onboarding screen. It prompts you to add your baby's information before monitoring can begin.



Baby's First Name - enter your baby's name.
Birthdate - select from the date picker.
Gender (optional) - choose from the dropdown.
Notes (optional) - add any health conditions, allergies, or preferences the AI should know about from the very start.
Tap 'Start Monitoring' when ready. You will be taken straight to the Home Dashboard.
Note: You can add or update notes at any time from the Profile page.

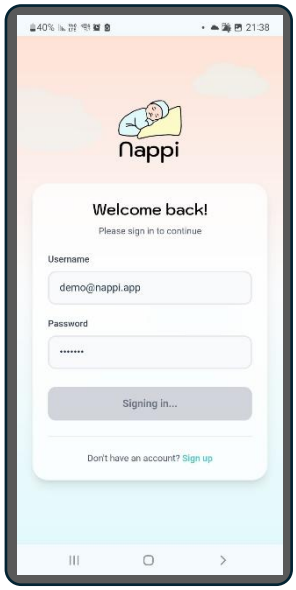
2.4. Signing In

The Sign In page asks for your username (email address) and your password. After entering your credentials, tap the 'Sign In' button.

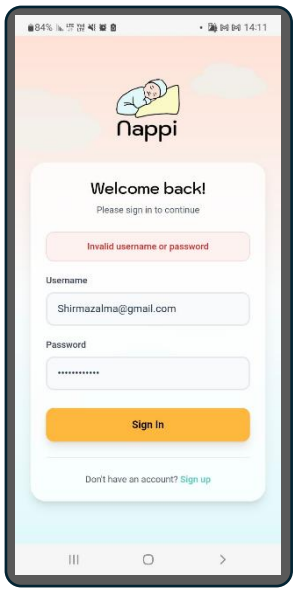


Username - the email address you registered with.
Password - the password you chose when you signed up.
Don't have an account? Tap 'Sign up' at the bottom to register.

Once you tap 'Sign In', the button changes to 'Signing in…' while the app verifies your credentials.

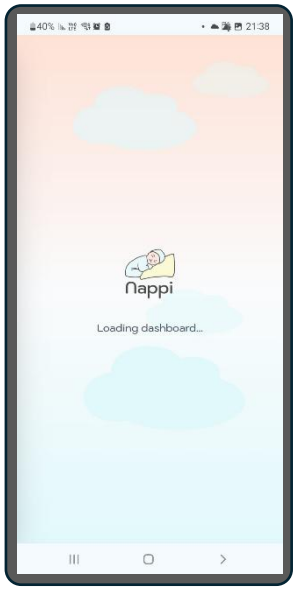


Invalid Credentials
If your username or password is incorrect, the app will display an 'Invalid username or password' error banner in red at the top of the form. The fields remain filled so you can correct them and try again.



2.5. Loading the Dashboard

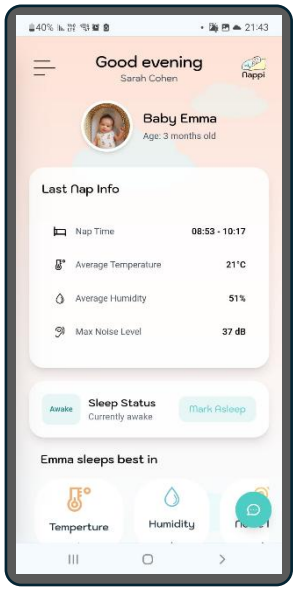
After successful authentication, Nappi displays a full-screen loading animation while it fetches your data. You will see the Nappi logo and the text 'Loading dashboard…'



This usually takes a few seconds. Once complete, you will arrive at the Home Dashboard.

3. Home Dashboard

The Home Dashboard is the heart of Nappi. It greets you with a time-appropriate message ('Good morning', 'Good afternoon', or 'Good evening') followed by your name. Your baby's name, and age are shown prominently at the top.



3.1. Last Nap Info

The 'Last Nap Info' card shows a summary of the most recent sleep session recorded. It includes:

Nap Time - start and end time of the last nap (e.g. 15:51 - 15:54).

Average Temperature - the average room temperature during that nap (°C).

Average Humidity - the average humidity level during the nap (%).

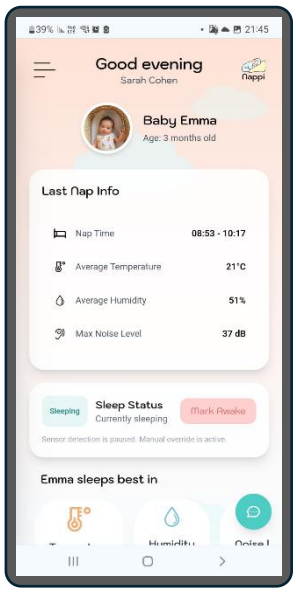
Max Noise Level - the loudest noise detected during the nap (dB).

3.2. Sleep Status & Manual Control

Below the Last Nap card is the Sleep Status card. It shows whether your baby is currently 'Awake' or 'Sleeping' based on the system readings and provides a button to manually override the sensor-detected state.

Mark Asleep - tap this when your baby fell asleep to start tracking a nap.

Mark Awake - tap this once your baby wakes up.



Note: When you manually override the sleep status, you will see the message 'Sensor detection is paused. Manual override is active.' Automatic detection resumes once the override timeout is cleared.

3.3. Sleeps Best In

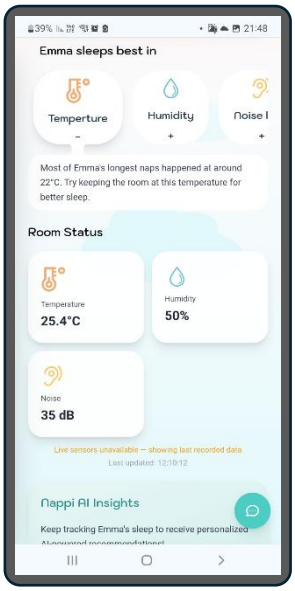
Scroll down to find the 'Sleeps Best In' section. This shows the optimal room conditions learned for your baby. There are three functional condition cards that you can scroll horizontally:

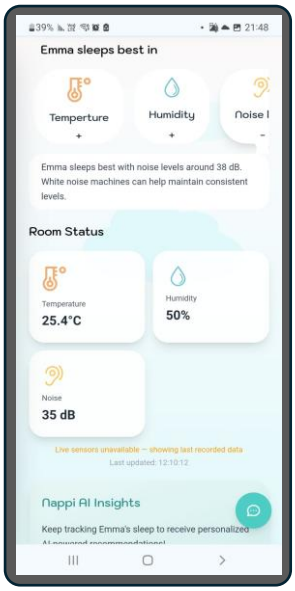
Temperature - tap to expand and read the optimal temperature tip.

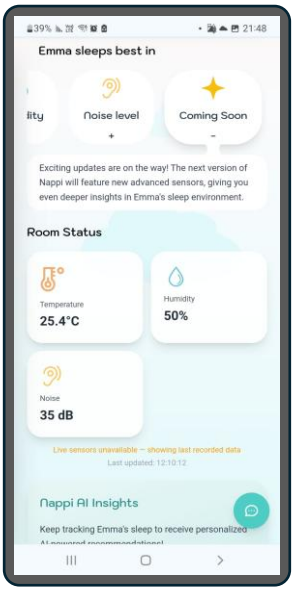
Humidity - tap to expand and read the optimal humidity recommendation.

Noise Level - tap to expand and read the ideal noise environment suggestion.

Coming Soon - a placeholder card for future sensor types. Tapping it reveals a note that advanced sensors are being added in the next version.





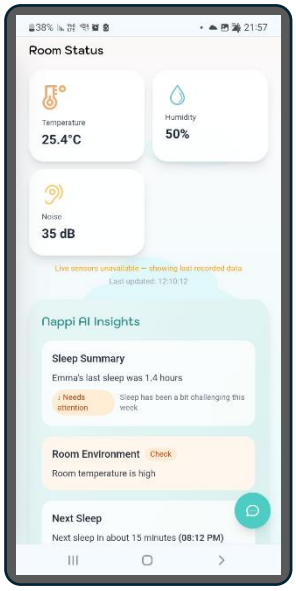


Note: The tips shown in each bubble are personalized based on your baby's historical sleep data.

3.4. Room Status

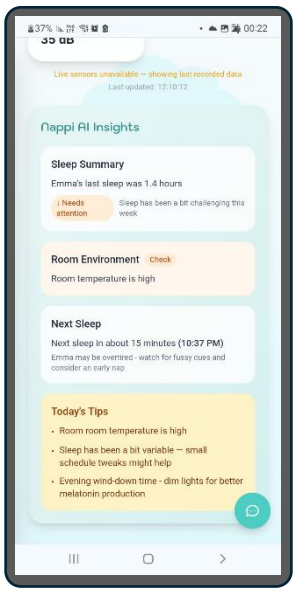
The Room Status section displays the current sensor readings from your baby's room. Three tiles show the current Temperature (°C), Humidity (%), and Noise (dB).

If the sensors are temporarily unavailable, the app shows the last recorded reading with an orange notice at the bottom: 'Live sensors unavailable - showing last recorded data', along with the exact timestamp of the last reading. A '--' symbol in a tile means that particular sensor has no reading at all for the current period.



3.5. Nappi AI Insights

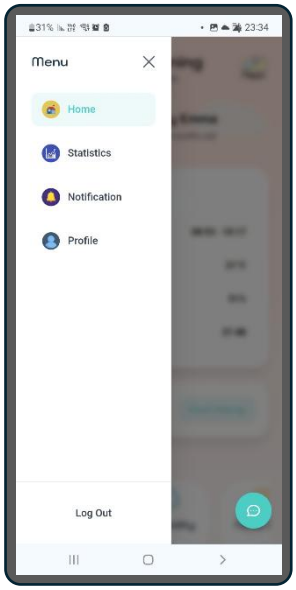
At the bottom of the dashboard, you will find the 'Nappi AI Insights' section. This provides a short AI-generated summary of your baby's recent sleep and environment. When sufficient data has been collected, it includes personalized tips and pattern observations. Before enough data is gathered, it encourages you to keep tracking: 'Keep tracking ___'s sleep to receive personalized AI-powered recommendations!'



4. Navigating the App

4.1. The Burger Menu

A hamburger icon (three horizontal lines) appears in the top-left corner of every page. Tap it to open the side navigation menu.



The menu contains the following destinations:

Home - returns to the main dashboard.

Statistics - opens the Sleep Analytics page with charts.

Notification - opens your alert history.

Profile - opens your account and baby settings.

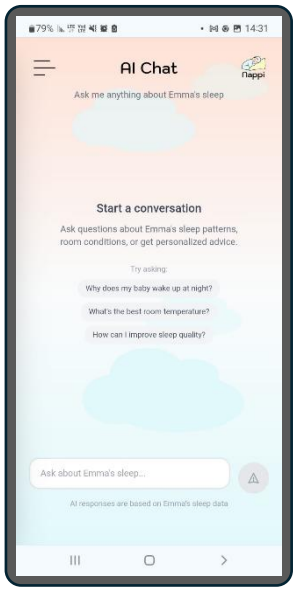
Log Out - signs you out of the app.

4.2. The AI Chat Button

A circular chat button is always visible in the bottom-right corner of every page. Tap it at any time to jump directly to the AI Chat page without opening the menu.

5. AI Chat

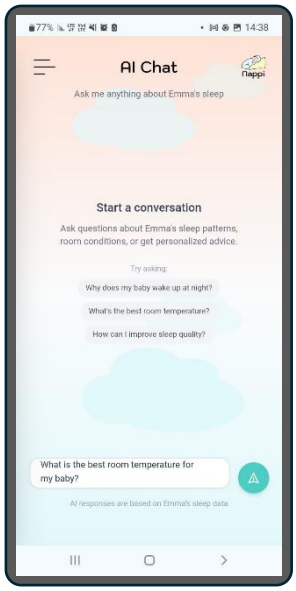
The AI Chat page lets you have a conversation with Nappi's AI assistant. It knows your baby's sleep history, room conditions, notes you have added in the Profile page and more context related data gathered and analyzed on the baby. You can ask it questions or request personalized advice.



5.1. Strating a Conversation

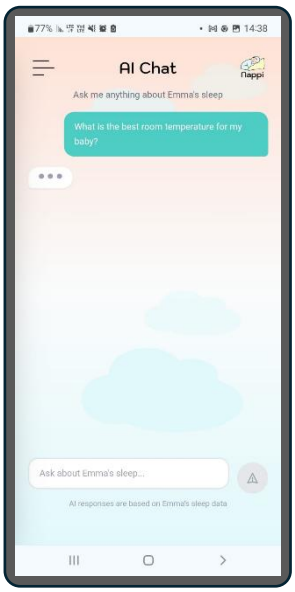
When you first open the chat, you will see an empty state with the title 'Start a conversation' and three suggestion prompts you can tap to get started quickly (pressing them would paste the question into the input field)

You can also type any question directly into the input field at the bottom and press Enter (or the send button) to submit it.



5.2. Reading Responses

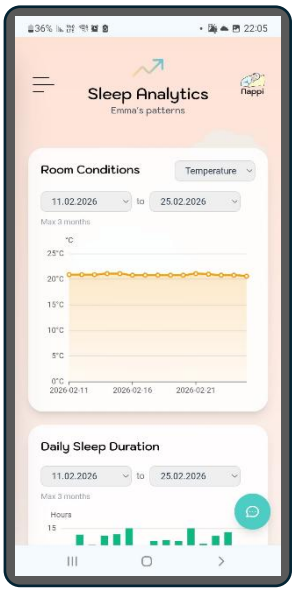
User messages appear in teal bubbles on right side of screen. AI's responses appear in white bubbles on the left.



The AI draws on everything it knows on your baby for a contextual, personalized reply.
Note: AI responses are generated by a language model. Please use them as guidance rather than medical advice. Always consult a healthcare professional for medical concerns.

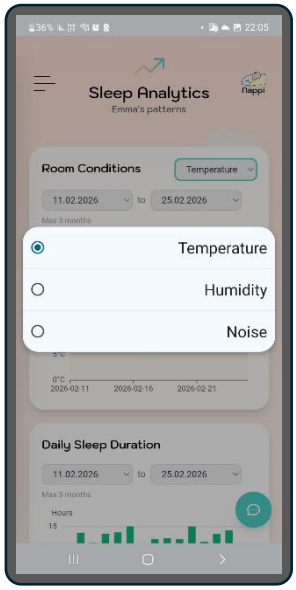
6. Sleep Analytics

The Sleep Analytics page (accessible from the menu as 'Statistics') provides detailed charts and AI analysis of your baby's sleep history.

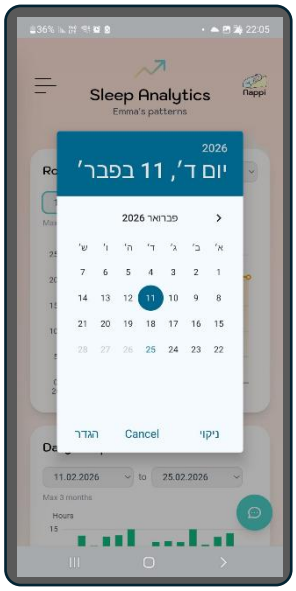


6.1. Room Conditions Chart

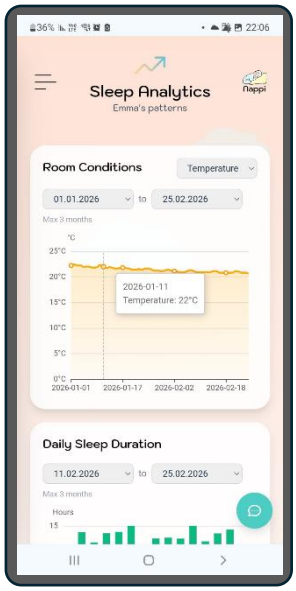
The first chart tracks room environment over a selected date range. Use the dropdown at the top-right to switch between sensor types:
Temperature (°C) - displayed as an orange line.
Humidity (%) - displayed as a blue line.
Noise (dB) - displayed as a green line.



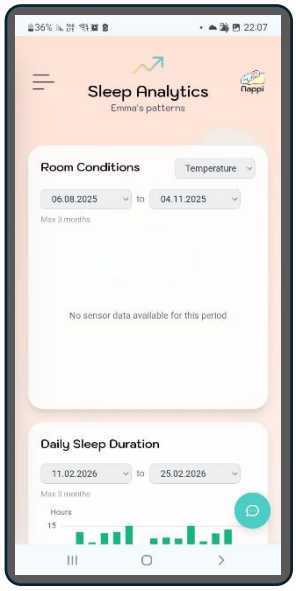
Set a custom date range using the two date pickers below the chart title. The maximum allowed range is 3 months (90 days).



To view the exact value for any specific date, simply tap or click on that point in the chart. A tooltip will appear showing the date and the precise measurement.



If you select a date range for which no sensor readings exist, the chart area will display the message 'No sensor data available for this period' instead of a graph. Try adjusting the date range to a period when the sensor was active.



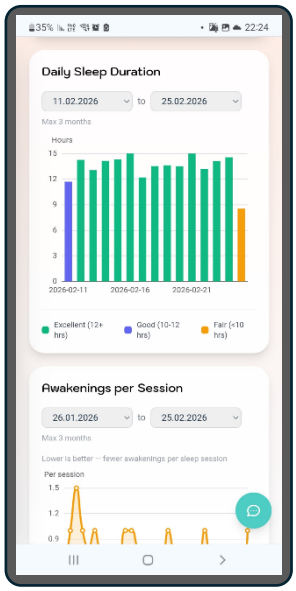
6.2. Daily Sleep Duration Chart

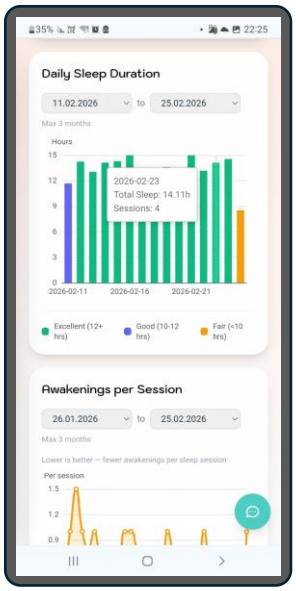
Below the Room Conditions chart is the Daily Sleep Duration bar chart. Each bar represents a day, and the height corresponds to the total hours of sleep recorded on that day. Bars are color-coded by quality:

Green - Excellent sleep (12 or more hours).

Blue / Purple - Good sleep (10–12 hours).

Orange - Fair sleep (less than 10 hours).

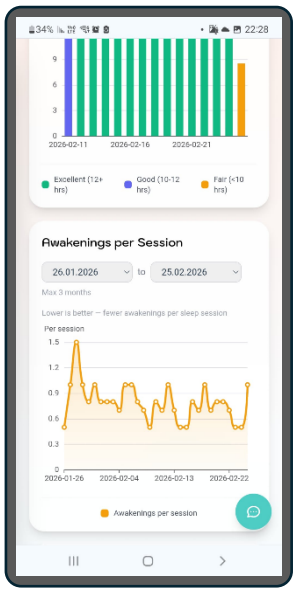




This chart also supports a 3-month maximum date range. Use the date pickers to focus on the period you care about most.

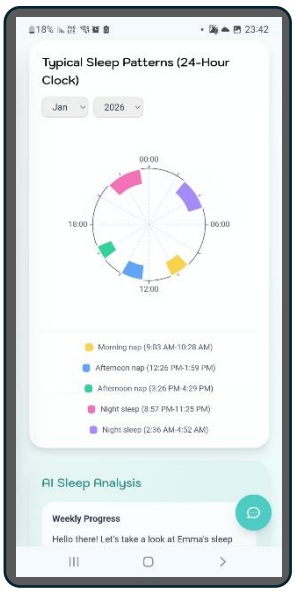
6.3. Awakenings per Session Chart

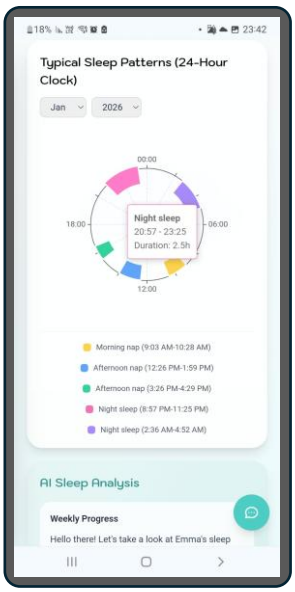
Below the Daily Sleep Duration chart is the Awakenings per Session line chart. Each data point represents the average number of times the baby woke up per sleep session on that day.



6.4. Typical Sleep Patterns (24-Hour Clock)

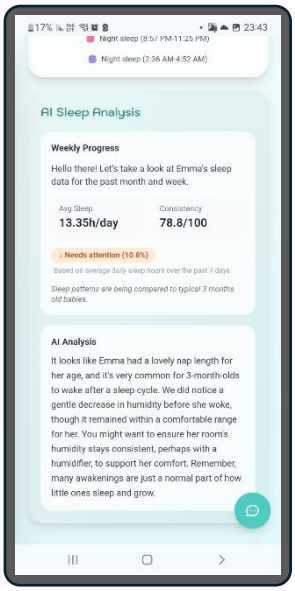
The third visualization is a polar (circular) chart showing the times of day when your baby most commonly sleeps. Each line segment represents a typical sleep window, labelled in the legend below. Simply choose a month and a year from the dropdown icons.





6.5. AI Sleep Analysis

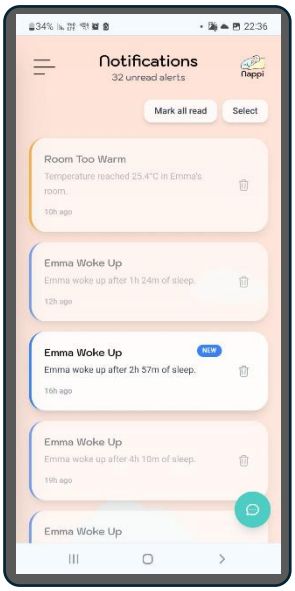
At the bottom of the Statistics page is the AI Sleep Analysis section, split into two parts:
Weekly Progress - Shows the average daily sleep duration, a consistency score out of 100, and a trend indicator (e.g. 'Improving'). Patterns are benchmarked against typical data for a baby of the same age.



AI Analysis - A paragraph of personalized commentary explaining the sleep data and offering actionable suggestions related to last week data and general context.

7. Notifications

The Notifications page shows a full history of alerts sent by Nappi. Access it from the burger menu under 'Notification'. When all notifications have been seen, the page shows 'All caught up!' below the title.



7.1. Types of Notifications

Nappi sends several kinds of alerts, each with a distinctive icon label on the left side of the card:

AW (Awake) — your baby woke up, showing the exact time and nap duration.

N (Noise) — a noise spike was detected in the room, worth checking on.

T (Temperature) — a significant temperature change occurred.

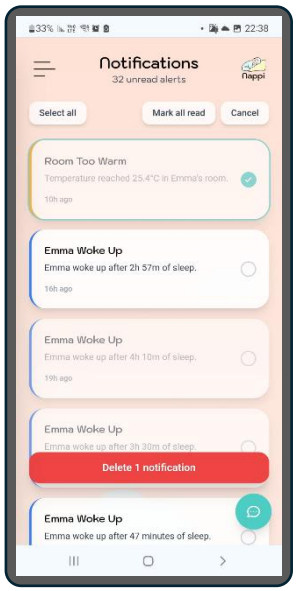
H (Humidity) — a notable humidity change was detected.

7.2. Deleting Individual Notifications

Each notification card shows the alert type, a description, and how long ago it was received. To delete a single notification, tap the trash icon on the right side of the card.

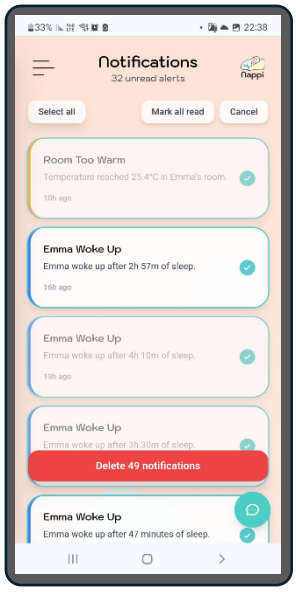
7.3. Bulk Deleting Notifications

For removing multiple notifications at once, tap the 'Select' button in the top-right corner. The interface switches to selection mode: each card gains a checkbox on the right, a 'Select all' button appears top-left, and a 'Cancel' button appears top-right.



Tap individual cards to check or uncheck them. A red 'Delete X notification(s)' bar appears at the bottom showing how many you have selected. Tap it to permanently remove those notifications.

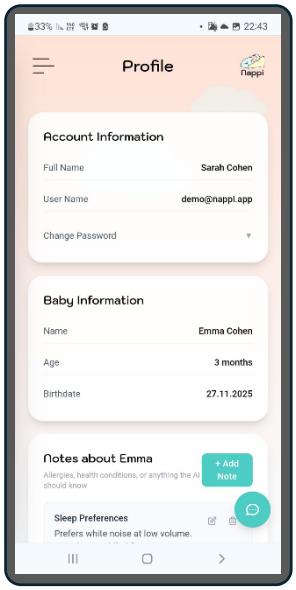
Tap 'Select all' to instantly check every notification on the page. The delete bar will update to show the full count.



Tap 'Cancel' at any time to exit selection mode without deleting anything.

8. Profile

The Profile page lets you view and manage all information about your account and your baby. Open it from the burger menu under 'Profile'.

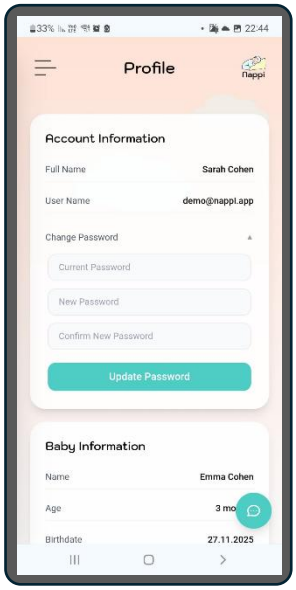


8.1. Account Information

The Account Information card shows your full name and username (email address). It also contains the Change Password section.

Changing Your Password

Tap the 'Change Password' row to expand a password-change form directly within the card.



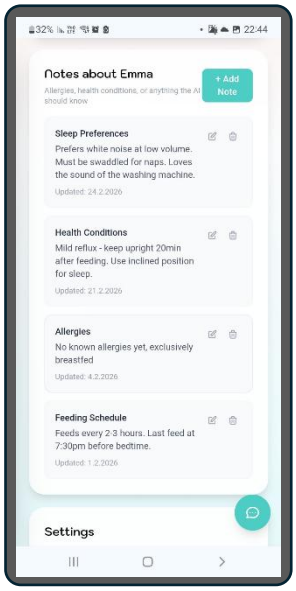
Current Password - enter your existing password for verification.
New Password - choose your new password.
Confirm New Password - type the new password again to confirm.
Tap 'Update Password' to save the change.

8.2. Baby Information

Scrolling down reveals the Baby Information card, showing your baby's name, age (in months), and birth date. This data was entered during registration.

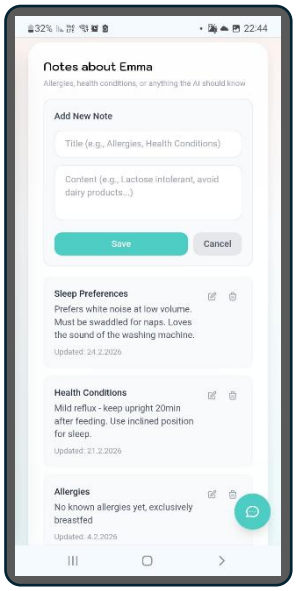
8.3. Notes About Your Baby

The Notes section is where you add any information the AI should know about your baby - allergies, health conditions, feeding schedules, sleep preferences, and so on. Notes appear as individual cards, each with a title, content, last-updated date, and edit / delete icons.



<u>Adding a New Note</u>

Tap the '+ Add Note' teal button at the top-right of the Notes section. An 'Add New Note' form will expand inline, with two fields:



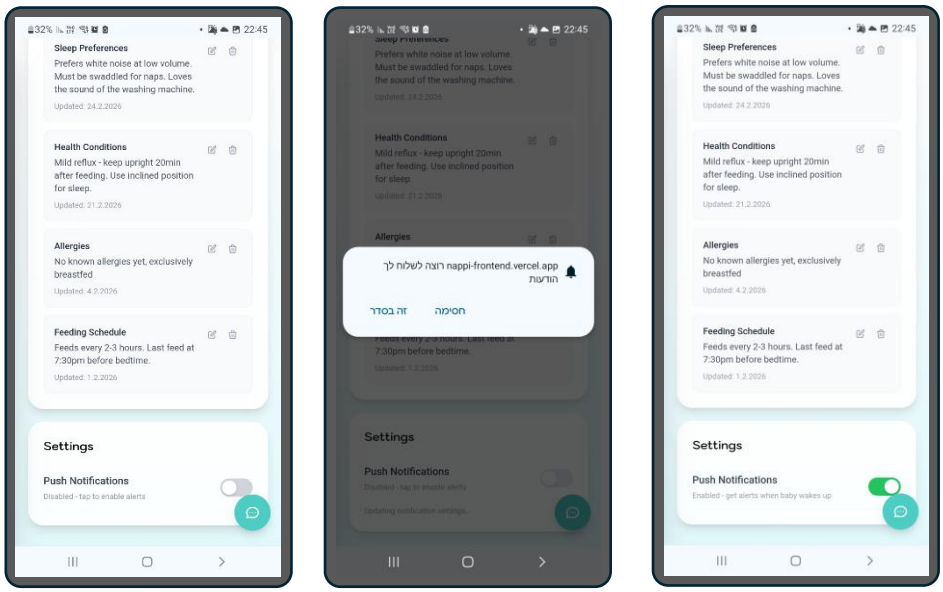
Title - a short label for the note, e.g. 'Allergies', 'Health Conditions', 'Feeding Schedule'.
Content - the full details, e.g. 'Lactose intolerant, avoid dairy products'.

Tap 'Save' to add the note, or 'Cancel' to discard it. New notes appear immediately in the list below the form.
Note: The AI reads all your notes every time it generates insights or responds to a chat message. Keeping notes up to date leads to better, more tailored advice.

Editing and Deleting Notes
Each existing note has a pencil (edit) icon and a trash (delete) icon. Tap the pencil icon to open the note in edit mode and update its contents. Tap the trash icon to permanently remove the note.

8.4. Settings

Scrolling to the bottom of the Profile page reveals the Settings section.



Push Notifications - A toggle switch that enables or disables browser push notifications. When enabled, the label reads 'Enabled - get alerts when baby wakes up'. Toggle it off if you want to pause push alerts without losing your notification history.
Note: Enabling push notifications requires your browser's permission. If you previously denied permission, you may need to update your browser settings to allow notifications from the Nappi website.

Maintenance Guide

1. Introduction

This document serves as a comprehensive Maintenance Guide for the Nappi Baby Sleep Monitoring System. It is intended for developers and maintenance personnel responsible for operating, updating, and extending the system after the official project hand-off.

Nappi is a baby sleep monitoring PoC that allows parents to track their baby's sleep sessions, monitor room environment conditions (temperature, humidity, noise), and receive AI-powered insights. The system consists of a backend API server, a M5Stack controller with sensors, a frontend web application, and a cloud-hosted PostgreSQL database.

This manual walks you through every feature of the app, from your first registration all the way to reading AI sleep analysis charts. Screenshots are included throughout to guide you step by step.

2. System Overview

The Nappi system is composed of four main components:

- **Backend API (FastAPI + Python):** Central service handling data management, sensor data collection, AI analysis, real-time alerts, and all business logic. Located in the backend directory.
- **Hardware Sensor Unit (M5Stack + Sensors):** Embedded device deployed in the baby's room responsible for continuous environmental monitoring - collecting temperature, humidity, and noise level readings and transmitting them to the backend. Located in the M5Stack directory.
- **Frontend Web Application (React + TypeScript):** User-facing interface for viewing sleep data, managing baby profiles, chatting with the AI assistant, and receiving notifications. Located in the frontend directory.
- **Database (PostgreSQL on Neon):** Cloud-hosted serverless PostgreSQL database storing all persistent data including sleep records, sensor readings, alerts, and user information.

Communication between frontend and backend occurs via RESTful HTTP endpoints and Server-Sent Events (SSE) for real-time alert streaming.

3. System Environment

3.1. Software Requirements

Component	Requirement	Notes
Python	3.10 or higher	Required for backend. asyncpg and modern async features require 3.10+
Node.js	18.x or 20.x+	Required for frontend build and development
npm	9.x+	Comes with Node.js
PostgreSQL	15+ (Neon serverless)	Cloud-hosted. no local installation required

Git	Latest stable	For version control and deployment
pip	Latest stable	Python package manager
Operating System	macOS, Linux, or Windows (WSL recommended)	Development and production
Web Browser	Chrome, Firefox, Safari, or Edge (modern versions)	For accessing the frontend application

3.2. External Services

Service	Purpose	Required Credentials
Neon PostgreSQL	Cloud database hosting	Connection string (DB_CONNECTION_STRING)
Google Gemini AI	AI-powered sleep insights and chat	API key (GEMINI_API_KEY)
Web Push (VAPID)	Browser push notifications	VAPID public/private key pair + email

3.3. Network Requirements

- The backend server runs on port 8000
- The frontend development server runs on port 5173
- The sensor data source (M5Stack controller) communicates via HTTP on port 8001 (default address: 192.168.117.254, configurable via SENSOR_API_BASE_URL env var)
- CORS is configured to allow requests from:
  - localhost:5173
  - 127.0.0.1:5173
  - localhost:3000

  Additional origins can be added via the CORS_EXTRA_ORIGINS environment variable

4. Installation Instructions

4.1. Backend Installation

Step 1: Prerequisites

Before installing the backend, ensure you have:

- Python 3.10+ installed
- pip package manager
- Access to the Neon PostgreSQL database (connection string)
- Google Gemini API key
- VAPID keys for web push notifications (optional, for push notification support)

Step 2: Clone and Install Dependencies

Clone the backend repository(from submission repo):

Open a new ‘backend’ folder and enter it, inside of it open cmd and run commands:

```
git clone --no-checkout https://github.com/oferElz/Nappi.git backend
cd backend
git sparse-checkout init --cone
git sparse-checkout set Code/Backend
git checkout main
cd Code/Backend
```

Create a Python virtual environment and activate it:

```
python3 -m venv .venv
source .venv/bin/activate    # macOS/Linux
.venv\Scripts\activate       # Windows
```

Install all dependencies:

```
pip install -r requirements.txt
```

The requirements include: fastapi, uvicorn[standard], pydantic>=2.0, aiohttp, apscheduler, python-dotenv, sqlalchemy[asyncio], asyncpg, certifi, google-genai>=1.0, pytz, pywebpush.

Step 3: Configure Environment Variables

Create a .env file in the backend directory with the following variables:

```
DB_CONNECTION_STRING=postgresql+asyncpg://<user>:<password>@<host>/<database>
GEMINI_API_KEY=<your-gemini-api-key>
VAPID_PUBLIC_KEY=<your-vapid-public-key>
VAPID_PRIVATE_KEY=<your-vapid-private-key>
VAPID_EMAIL=<admin-email-for-vapid>
```

IMPORTANT: Never commit the .env file to version control. It contains sensitive credentials.

In the current deployment, we configured the environment variables directly in Render (by copying the values from our .env file).

Step 4: Database Setup

The system uses Neon PostgreSQL (serverless). All tables reside under the "Nappi" schema (case-sensitive, must be double-quoted in queries).

If setting up a fresh database, run the migration SQL files in order:

```
psql "<your-connection-string>" -f migrations/001_create_alerts_table.sql
psql "<your-connection-string>" -f migrations/002_add_baby_notes.sql
psql "<your-connection-string>" -f migrations/003_remove_anomalies_column.sql
```

```
psql "<your-connection-string>" -f migrations/004_drop_sleep_quality_score.sql
psql "<your-connection-string>" -f migrations/005_drop_heart_rate_and_babies_notes.sql
```

To seed demo data for development/testing, run:

```
python -m app.db.seed_demo_data
```

Step 5: Start the Backend

From the backend directory with the virtual environment activated:

```
uvicorn app.main:app --reload --port 8000
```

The backend API should now be accessible at http://localhost:8000. API documentation is automatically available at http://localhost:8000/docs (Swagger UI).

4.2. Frontend Installation

Step 1: Prerequisites

Ensure you have Node.js 18.x or 20.x+ and npm installed.

Step 2: Clone and Install Dependencies

Clone the frontend repository (from submission repo):

Open a new ‘frontend’ folder and enter it, inside of it open cmd and run the commands::

```
git clone --no-checkout https://github.com/oferElz/Nappi.git nappi-frontend
cd nappi-frontend
git sparse-checkout init --cone
git sparse-checkout set Code/Frontend
git checkout main
cd Code/Frontend
```

Install all dependencies:

```
npm install
```

Step 3: Configure API URL

The API base URL is configured in src/api/client.ts. By default, it falls back to http://localhost:8000. For production deployment, set the VITE_API_URL environment variable (in a .env file in the frontend root):

```
VITE_API_URL=https://your-production-api.com
```

If VITE_API_URL is not set, the frontend will use http://localhost:8000 as the default.

Step 4: Start the Frontend

For development:

```
npm run dev
```

The frontend should now be accessible at http://localhost:5173.

For production build:

```
npm run build
```

This generates optimized static files in the dist/ directory that can be served by any static file server (Nginx, Vercel, Netlify, etc.).

4.3. Running Both Components Together

For a complete development environment, run both services simultaneously:

Terminal 1 (Backend):

```
cd backend
source .venv/bin/activate
uvicorn app.main:app --reload --port 8000
```

Terminal 2 (Frontend):

```
cd nappi-frontend
npm run dev
```

Access the application at http://localhost:5173.

5. Backend Structure

The backend project follows the directory structure shown below:

```
backend/
+-- app/
|   +-- main.py                  # FastAPI app entry point, CORS, lifespan
|   +-- api/                     # API route handlers
|   |   +-- alerts.py            # Alert & push notification endpoints
|   |   +-- auth.py              # Authentication endpoints (signup, signin)
|   |   +-- babies.py            # Baby notes CRUD endpoints
|   |   +-- chat.py              # AI chat endpoint
|   |   +-- endpoints.py         # Dashboard endpoints (last sleep, room metrics)
|   |   +-- models.py            # Pydantic request/response models
|   |   +-- sensor_events.py     # Sleep start/end, intervention endpoints
|   |   +-- stats.py             # Statistics & analytics endpoints
|   +-- core/                    # Core infrastructure
|   |   +-- constants.py         # Application constants
|   |   +-- database.py          # DatabaseManager singleton (async SQLAlchemy)
|   |   +-- settings.py          # Settings class (env vars, thresholds, config)
|   |   +-- utils.py             # Sensor mapping utilities
|   +-- services/                # Business logic services
|   |   +-- alert_service.py     # AlertService + SSEManager (real-time alerts)
|   |   +-- auth_manager.py      # AuthManager (signup, signin, password)
|   |   +-- babies_data.py       # BabyDataManager (central DB access layer)
|   |   +-- chat_service.py      # ChatService (Gemini AI chat with context)
|   |   +-- correlation_analyzer.py  # Awakening cause analysis
|   |   +-- daily_summary.py     # Daily summary generation
```

```
|   |   +-- data_miner.py        # HTTP sensor data source (M5 hardware)
|   |   +-- optimal_stats.py     # Optimal conditions calculator
|   |   +-- push_service.py      # Web push notification service
|   |   +-- schedule_predictor.py # Next sleep prediction
|   |   +-- scheduler.py         # APScheduler setup
|   |   +-- sleep_patterns.py    # Sleep pattern clustering
|   |   +-- sleep_state.py       # SleepStateManager (in-memory state)
|   |   +-- tasks.py             # Background scheduled tasks
|   |   +-- trend_analyzer.py    # 7/30-day trend analysis
|   +-- utils/                   # Utility modules
|   |   +-- sleep_blocks.py      # Sleep block grouping algorithm
|   +-- db/                      # Database utilities
|       +-- generate_models.py   # DB model generator
|       +-- models.py            # Auto-generated Pydantic DB models
|       +-- seed_demo_data.py    # Demo data seeder
+-- migrations/                  # SQL migration files (run manually)
+-- requirements.txt             # Python dependencies
+-- .env                         # Environment variables (not in git)
```

Description of each directory:

- **api**: Defines all REST API endpoints organized by feature domain. Each file registers a FastAPI router with specific routes. models.py contains all Pydantic models for request/response validation.
- **core**: Infrastructure-level components. database.py provides the async database connection manager. settings.py centralizes all configuration. utils.py maps sensor names to API endpoints and DB columns.
- **services**: Contains all business logic. Each service is a singleton accessed via get_*() factory functions. Services handle AI integration, data analysis, alert management, and background task execution.
- **utils**: Shared utility functions. sleep_blocks.py groups raw awakening events into consolidated sleep sessions.
- **db**: Database schema models and seeding tools for development.
- **migrations**: Manual SQL migration files to be executed sequentially against the database.

6. Frontend Structure

The frontend project follows the directory structure shown below:

```
nappi-frontend/
+-- src/
|   +-- main.tsx                 # Application entry point
|   +-- App.tsx                  # React Router setup, route definitions
|   +-- constants.ts             # Centralized constants (thresholds, timeouts,
limits)
|   +-- api/                     # API client modules
|   |   +-- client.ts            # Axios instances (standard + chat with 60s timeout)
|   |   +-- alerts.ts            # Alert & push API functions
```

```
|   |   +-- auth.ts              # Auth + baby notes API functions
|   |   +-- chat.ts              # AI chat API function
|   |   +-- room.ts              # Room metrics API function
|   |   +-- sleep.ts             # Sleep summary & status API functions
|   |   +-- stats.ts             # Statistics & analytics API functions
|   +-- pages/                   # Page-level components
|   |   +-- HomeDashboard.tsx    # Main dashboard (sleep status, room, AI summary)
|   |   +-- Statistics.tsx       # Charts, trends, patterns, insights
|   |   +-- Chat.tsx             # AI chat interface
|   |   +-- Notifications.tsx    # Alert history with real-time SSE
|   |   +-- UserProfile.tsx      # User info, baby notes, push settings
|   |   +-- Login.tsx            # Login form
|   |   +-- Signup.tsx           # Registration form
|   |   +-- Welcome.tsx          # Welcome/landing page
|   |   +-- Onboarding.tsx       # Baby registration flow
|   +-- components/              # Reusable UI components
|   |   +-- Layout.tsx           # App shell (navigation, gradient background)
|   |   +-- LayoutContext.tsx    # Shared layout state context
|   |   +-- ProtectedRoute.tsx   # Authentication guard
|   |   +-- Btn.tsx              # Reusable styled button
|   |   +-- ChatFloatingButton.tsx # Floating chat action button
|   +-- hooks/                   # Custom React hooks
|   |   +-- useAlerts.ts         # SSE connection for real-time alerts
|   +-- types/                   # TypeScript type definitions
|   |   +-- auth.ts              # User, Baby, auth request/response types
|   |   +-- metrics.ts           # Sleep, room, stats, AI response types
|   +-- utils/                   # Utility functions
|   |   +-- session.ts           # Session cookie management
|   +-- styles/                  # CSS/styling files
+-- public/                      # Static assets
+-- package.json                 # Dependencies and scripts
+-- tailwind.config.js           # Tailwind CSS configuration
+-- vite.config.ts               # Vite build configuration
+-- tsconfig.json                # TypeScript configuration
```

Description of each directory:

- **api**: Axios-based API client modules. client.ts creates two Axios instances: a standard one (10s timeout) and a chat-specific one (60s timeout for AI responses). Other files wrap specific API calls.
- **pages**: Full-page React components, one per route. Each page manages its own local state with useState/useEffect.
- **components**: Shared UI components used across multiple pages. ProtectedRoute checks session cookies to guard authenticated routes.
- **hooks**: useAlerts manages the SSE connection to /alerts/stream for real-time alert delivery with auto-reconnect.

- **types**: TypeScript interfaces matching the backend Pydantic models.
- **utils**: session.ts manages the nappi_user session cookie
  (set on login, cleared on logout or browser close).

7. Maintenance Tasks

7.1. Backend Maintenance

Updating Python Dependencies

From the backend directory with the virtual environment activated:

```
pip install --upgrade -r requirements.txt
```

IMPORTANT: Always test the application after upgrading dependencies, as version incompatibilities may occur. Pay special attention to pydantic, sqlalchemy, and google-genai updates.

Database Migrations

When schema changes are needed, create a new numbered SQL file in migrations/ (e.g., 006_description.sql) and run it against the database:

```
psql "<connection-string>" -f migrations/006_description.sql
```

Always quote the "Nappi" schema name in all SQL queries.

Gemini API Key Rotation

Update the GEMINI_API_KEY in the .env file and restart the backend. The Gemini client is lazy-loaded and will pick up the new key on next initialization.

VAPID Key Rotation

If VAPID keys need to be rotated, update both VAPID_PUBLIC_KEY and VAPID_PRIVATE_KEY in .env file.
Note: existing push subscriptions will become invalid and users will need to re-subscribe.

Monitoring Background Tasks

The APScheduler runs three periodic jobs:

1. Sensor data collection (every 5 seconds) -- only active when babies are sleeping
2. Daily summary generation (10:00 Asia/Jerusalem daily)
3. Optimal stats calculation (10:05 Asia/Jerusalem daily)

Check server logs for any scheduler errors. The scheduler starts automatically with the FastAPI application lifespan.

Important: SleepStateManager is In-Memory

The SleepStateManager tracks currently sleeping babies in memory. If the backend server restarts, all active sleep sessions are lost. Sleeping babies will need to re-trigger a sleep-start event after a restart.

7.2. Frontend Maintenance

Updating npm Dependencies

```
npm update
```

Or for major version upgrades:

```
npm install <package>@latest
```

Changing the Backend API URL

For production deployment, set the VITE_API_URL environment variable in a .env file at the frontend root:

```
VITE_API_URL=https://your-production-api.com
```

This is read by src/api/client.ts via import.meta.env.VITE_API_URL, with http://localhost:8000 as the default fallback.

CORS Configuration

When deploying to a new domain, either add the domain to CORS_ORIGINS in the backend app/core/settings.py file or set the CORS_EXTRA_ORIGINS environment variable with comma-separated additional origins.

Building for Production

```
npm run build
```

Caregiver Persona Survey

Persona name: Naama
Age: 39
Baby's age: 11 months
Number of children: 2
Background: Second baby has had significant and ongoing sleep difficulties since birth. Familiar with sleep tracking from her first child but finds this baby's needs much harder to read and manage.

Question: What do you currently do to track or manage your baby's sleep, and why?
Answer: With my first baby I used a basic baby monitor like everyone have, to monitor the baby safety, and to see when she wakes up. My first baby was easily falling asleep and so that was my daily use of the monitor. With this one I've been having hard time ever since she was born. I tried to track her sleep sessions on notes in my phone so I will see if it get better but it was very inconvenient, so I stopped tracking it.

Question: Have you ever noticed a connection between the room environment and how well your baby sleeps?
Answer: Yes, with temperature. On the baby monitor screen I can see the temperature in her room, most of the nights the value is around 23, and many times when I looked over the baby monitor and saw 22 or 21, a few minutes later she woke up and started crying. I don't really know if it has any connections but it did happen many times, to the point where change in the values in the baby monitor screen caused me to be 'ready' for her to wake up.

Question: What information do you wish you had after a bad nap or a rough night?
Answer: I want to know what was different. right now, it's just the feeling that something maybe make her uncomfortable but I have nothing to point to. I think a full scale information could have helped, its hard to see patterns while living daily life of work-parenting. From heart rate, body temperature, room conditions, development stage and the appropriate related sleep issues.

Question: If you had to create a tool that could monitor your baby in real time, what would you want it to show you?
Answer: Temperature for sure, I think humidity is also relevant, both of my daughters have skin issues and rashes, sometimes I add a humidifier to the bedroom to help them ease the itchiness. Maybe also heart rate and body temperature.

Question: Would you want to receive an alert on your phone when the baby wakes up, or does that feel like too much?
Answer: I'd want it when I'm not fully focused. If I've stepped away to eat or shower and I'm not sure if she's still sleeping, yes absolutely.

Question: Would you want to see charts of your baby's sleep over time like days, weeks?
Answer: I want it, but it has to be simple. If it shows me that the last two weeks were better or worse than the two before and gives me a possible reason why, that's enough.

Question: Is there information you'd want that goes beyond the room environment, something deeper about the baby herself during sleep?
Answer: Breathing and movement. I want to know she's breathing normally and that she hasn't rolled into a bad position.

Question: How comfortable are you with the idea of an AI analyzing your baby's sleep data and offering you suggestions?
Answer: If it's based on my baby's actual data and not just generic advice, I'm open to it. But it is important that the information the AI have access to, would be not personal, I don't want images or personal health issue to be public.

Question: Would you be comfortable with an AI chat that you could ask questions about your baby's sleep at any time?
Answer: Yes why not, its just a feature like in almost every app or website we have today. If the conversations with it would be like 'why might she be waking up more this week', then it is fine by me. What I would not like is if it would suggest medical actions when discussing my baby. I think it is important to limit the chat to be suggestions, and not acting as 'AI doctor'.

Question: Are there things you'd want the AI to do automatically without you having to ask, or do you prefer to stay in control?
Answer: I want it to proactively tell me if something looks off, like if the room has been consistently warmer than what's good for her sleep. I'd want to know that. But I don't want it acting on anything, contacting anyone, or making decisions. It should inform me and let me decide what to do.

Question: Would you feel comfortable if the app shared your baby's sleep data with your pediatrician automatically?
Answer: No. I would want to choose what to share and when. If I've had a hard week and I want to bring the charts to the appointment, that should be my decision. Automatic sharing feels like a loss of control.

Question: If you could only have one feature from a sleep monitoring tool, what would it be?
Answer: A summary after every nap that tells me what was my baby information during that nap and what was the room conditions like and whether that nap was shorter or longer than her usual.